

This is an important point to keep in mind: PostgreSQL relies on the underlying filesystem to implement persistence, and therefore tuning the filesystem is an important task in order to make PostgreSQL perform well. In particular, PostgreSQL stores all of its content (user data and internal status) in a single filesystem directory known as PGDATA. The PGDATA directory represents what the cluster is serving as databases, so it is possible for you to have a single installation of PostgreSQL and make it switch to different PGDATA directories to deliver different content. As you will see in the next sections, the PGDATA directory needs to be initialized before it can be used by PostgreSQL; the initialization is the creation of the directory structure within PGDATA itself and is, of course, a one-time operation.

The detailed contents of PGDATA will be explained in the next chapter, but for now, it will suffice for you to remember that the PGDATA directory is where PostgreSQL expects to find data and configuration files. In particular, the PGDATA directory is made up of at least the **Write-Ahead Logs (WALs)** and the **data storage**. Without either of those two parts, the cluster is unable to guarantee data consistency and, in some critical circumstances, even start.

WALs are a technology that many database systems use, and the basic idea of how they work is shared with other technologies like transactional filesystems (such as ZFS, UFS with Soft Updates, and so on). The idea is that, before applying any change to a chunk of data, an intent log will be made persistent. In this case, if the cluster crashes, it can always rely on the already-written intent log to understand what operations have been completed and what must be recovered (more details on this in later chapters). Please note that with the term “crash,” we refer to any possible disaster that can hit your cluster, including a software bug, but more likely a lack of electrical power, hard disk failures, and so on. PostgreSQL does commit to providing you with the best data consistency it can, and therefore, it makes a great effort to ensure that the intent log (WAL) is as safe as possible.

Internally, PostgreSQL keeps track of the tables’ structures, indexes, functions, and all the stuff needed to manage the cluster in its dedicated storage, the catalog.



The SQL standard defines a so-called **information schema**, a collection of tables common to all standard database implementations, including PostgreSQL, that the DBA can use to inspect the internal status of the database itself. For instance, the information schema defines a table that collects information about all the user-defined tables so that it is possible to query the information schema to see whether a specific table exists or not. The PostgreSQL catalog is what could be called an “information schema on steroids”: the catalog is much more accurate and PostgreSQL-specific than the general information schema, and the DBA can extract a lot more information about the PostgreSQL status from the catalog. Of course, PostgreSQL does support the information schema, but throughout the whole book, you will see references to the catalogs because they provide much more detailed information.

When the cluster is started, PostgreSQL launches a single process called the *postmaster*. The aim of the postmaster is to bootstrap the instance, spawning needed processes to manage the database activity, and then to wait for incoming connections. A user connection, often made over a TCP/IP connection, requires the postmaster to fork another process named the *backend process*, which in turn is in charge of serving one and only one connection.

This means that every time a new connection against the cluster is opened, the cluster reacts by launching a new backend process to serve it until the connection ends and the process is, consequently, destroyed. The postmaster usually also starts some utility processes that are responsible for keeping PostgreSQL in good shape while it is running; these processes will be discussed later, in this and the next chapters.

To summarize, PostgreSQL provides you with executables that can be installed wherever you want on your system and can serve a single cluster. The cluster, in turn, serves data out of a single PGDATA directory that contains, among other stuff, the user data, the cluster’s internal status, the catalog, and the WALs. Every time a client connects to the server, the postmaster process forks a new backend process that is the minion in charge of serving the connection.

From the concepts explained above, the following is a quick recap of the most complex terms used in PostgreSQL:

- **Cluster:** the whole PostgreSQL service.
- **Postmaster:** the first process the cluster executes, and this process is responsible for keeping track of the activities of the whole cluster. The postmaster spawns a backend process every time a new connection is established.
- **Database:** an isolated data container to which users (or applications) can connect. A cluster can handle multiple databases. A database can be made up of different objects, including schemas (namespaces), tables, triggers, and other objects you will see as the book progresses.
- **PGDATA:** the directory that, on persistent storage, is fully dedicated to PostgreSQL and its data. PostgreSQL stores the data within such a directory.
- **WALs:** the intent log of database changes, used to recover data from a critical crash.

Now that we've discussed the basic terminology related to PostgreSQL, it is time to get it installed on your machine.

Installing PostgreSQL

PostgreSQL can run on several operating systems, most notably Unix and Unix-like systems, including Linux, as well as on Microsoft Windows 11 or higher. So far, the most supported platform remains Linux because most PostgreSQL developers work on this platform, and so it is the one with the most tested use cases. However, deploying on other supported platforms should not present any problems and is not going to put your data at any risk.

This section will focus on installing PostgreSQL 16, since it is the latest stable version available worldwide. You will learn, however, how to build your own version of PostgreSQL, and this may also be the way for you to install other versions of PostgreSQL in the future.

Before installing PostgreSQL, you need to choose, or at least evaluate, how to install it. There are two main ways to get PostgreSQL up and running:

- Compiling from sources
- Using binary packages

Binary packages are provided by the PostgreSQL community or the operating system, and using them has the advantage that it can provide you with a smooth PostgreSQL installation.

Moreover, binary packages do not require a compilation toolchain and therefore are much easier to adopt. Lastly, a binary package adheres to the operating system conventions it has been built for (for instance, on where to place configuration files) and upgrades can be managed by the operating system as well. Since binary packages need to be pre-built from vendors, they may not reflect the latest released version. For example, when the PGDG delivers a new minor update, operating systems require some days to push out binary packages with such upgrades for all the supported platforms.

On the other hand, installing from sources requires a compilation toolchain, as well as much more time and CPU consumption to build PostgreSQL executables. You have full control over which components will be available in the final product and can trim and optimize your instance for very high performances and shrink resource consumption to a minimum. In the long term, however, you will be responsible for maintaining the installation and upgrading it in a similar manner.

What to install

PostgreSQL is split across several components to install:

- The PostgreSQL **server** is the part that can serve your databases to applications and users and is required to store your data.
- The PostgreSQL **client** is the library and client tool to connect to the database server. It is not required if you don't need to connect to the database on the very same machine, while it is required on client machines.
- The PostgreSQL **contrib** package is a set of well-known extensions and utilities that can enhance your PostgreSQL experience. This additional package is developed by the PGDG and is therefore well integrated and stable.
- The PostgreSQL **docs** is the documentation (e.g., man pages) related to the server and the client.
- PostgreSQL **PL/Perl**, **PL/Python**, and **PL/Tcl** are three components to allow the usage of programming languages—Perl, Python, and Tcl, respectively—directly within the PostgreSQL server.

The recommended set of components is the server, the client, and the **contrib** modules; these modules will be used across the book. You are free to decide whether to install the other components as you wish, but this book will not detail each of them.

Installing PostgreSQL from binary packages

In order to better understand the concepts explained in this book, we recommend readers try the code examples on their own; therefore, you will need a PostgreSQL instance available. While the best choice to get a full PostgreSQL instance at your fingertips is to install it on a virtual machine or a physical computer, we have also provided a set of Docker images as containerized PostgreSQL instances to run and experiment on. Therefore, you can choose between performing a full installation or a quick Docker setup to get a PostgreSQL machine ready. However, it is important for every DBA to be able to install PostgreSQL on several systems, and therefore, this section aims to show you how to perform a complete installation from scratch on a few Unix-like operating systems.

In the following sections, you will see how to install PostgreSQL on a few popular Linux and Unix operating systems, namely the following:

- Linux Docker containers
- GNU/Linux Debian, Ubuntu, and derivatives
- Fedora Linux (this also applies to Red Hat Enterprise Linux and compatible distributions, like Rocky Linux)
- FreeBSD

It is not possible to provide detailed instructions for every operating system out there, but the concepts presented in the following sections should prove insightful regardless.

Before getting to the practical installation, it is worth noting that binary packages could come in two flavors: those provided by the operating system vendor, and those provided by the PGDG. Usually, on Linux-based systems, you should use binary packages provided by the PGDG, because they are the most authoritative source for PostgreSQL. In fact, packages provided by the operating system vendor tend to become out of date very soon, which means they are usually a few versions behind the latest version globally available. On the other hand, on BSD platforms like FreeBSD, OpenBSD, and NetBSD, the operating system porters do an excellent job of keeping the packages provided by the operating system itself very up to date, so you can safely and easily use the operating system packages.

An important thing to note is that different operating systems store files in different places: usually, all the configuration files are placed within the PGDATA itself, but packages from some operating systems scatter the configuration files under the `/etc` directory. A few operating systems also place executables in specific paths, separated by the version of PostgreSQL, while others place all executables in the same path.

You need to investigate with the operating system package provider where each file or directory is placed in order to be able to configure and use PostgreSQL.

Using the book's Docker images

Docker is a container that allows you to run an isolated set of processes as if they are part of a micro virtual machine. The PGDG provides a Docker image that you can use to run a containerized cluster.

Explaining the Docker technology is out of the scope of this book, and in order to let you experiment in a quick and easy way with PostgreSQL, we have provided a set of Docker images, based on the PostgreSQL image, customized to let you experiment with the concepts explained in this book. You can use the above images as a starting point for your own projects, even if the above images are not meant to be used in a production environment. The images are contained in the `docker_images` directory of the book's code repository (<https://github.com/PacktPublishing/Learn-PostgreSQL-Second-Edition/>).

We separated every Docker image by means of the chapter the image refers to. There is a common *catch-all* image named *standalone* that can be used as a common base and will be used in the very first chapters. Other chapters, for instance, those on replication, require their own image to be executed.

In order to start the base *standalone* image, you can simply execute the shell script `run-pg-docker.sh`, as follows:

```
$ sh run-pg-docker.sh
...
postgres@learn_postgresql:~$
```

The script will ask you for a password; it is required that your user has `sudo` capabilities to connect the Docker network and ports. All the containers will launch a GNU Bash session with the `postgres` operating system user.

The first time each container is started, it will require some time because it needs to pull the PostgreSQL image from the network, install the needed packages, and configure the image. Ultimately, the system will push you to a Bash prompt; you are now logged in via the container as the user `postgres` and can start interacting with the system following the examples in this book.



In each container, the PGDATA directory is set to `/postgres/16/data`.

Once you leave the shell of the container, the container will stop and no more PostgreSQL-related processes will be active.

In order to start a specific *per-chapter* image, you can use the same script, specifying the chapter folder as an argument—for instance:

```
$ sh run-pg-docker.sh chapter_12_extensions
```

Every container will start with a pre-populated PostgreSQL instance, so that you can easily follow the code examples in every chapter.



Note: there might be some differences in the output you see in the code examples and the output you get from executing the same commands in a Docker container. For instance, automatically generated values and the tuple counting could be different, as well as timestamps and dates. Moreover, every Docker container will store data in a separate disk directory, therefore if you manipulate the contents of the containerized PostgreSQL instance, the next time you start the container your changes will have persisted.

Installing PostgreSQL on GNU/Linux Debian, Ubuntu, and derivatives

The PGDG provides binary packages for Debian and its derivatives, including the Ubuntu operating system family. In order to use the PGDG repositories, it is required for you to first install the source and signature of the repository:

```
$ sudo sh -c 'echo "deb http://apt.postgresql.org/pub/repos/apt $(lsb_release -cs)-pgdg main" > /etc/apt/sources.list.d/pgdg.list'

$ wget --quiet -O - https://www.postgresql.org/media/keys/ACCC4CF8.asc |
sudo apt-key add -

$ sudo apt-get update
```

This will ensure the repository sources for your operating system are up to date so that you can install the PostgreSQL packages:

```
$ sudo apt-get -y install postgresql
```

Debian and Ubuntu provide their own command to control the cluster, `pg_ctlcluster(1)`. The rationale for that is that on a Debian/Ubuntu operating system, every PostgreSQL version is installed in its own directory with separate configuration files, so there is a way to run different versions concurrently and manage them via the operating system. For example, configuration files are under the `/etc/postgresql/16/main` directory, while the data directory is set by default to `/var/lib/postgresql/16/main`.

If you want to enable PostgreSQL at boot time, you need to run the following command:

```
$ sudo update-rc.d postgresql enable

In order to start your cluster, you can use the service(1) command as
follows:

$ sudo service postgresql start
```

You have thus installed PostgreSQL on GNU/Linux Debian, Ubuntu, and derivatives.

Installing PostgreSQL on Fedora Linux

Fedora Linux PostgreSQL packages are provided by the PostgreSQL community. In order to allow `dnf(8)` to find PostgreSQL packages, you need to install the PGDG repository, and then proceed with the installation as a distribution package:

```
$ sudo dnf install -y https://download.postgresql.org/pub/repos/yum/
reporpms/F-38-x86_64/pgdg-fedora-repo-latest.noarch.rpm
```

The list of available repositories can be obtained from the PostgreSQL official website on the download page (see the *References* section).

Packages are named with the postfix of the version number. You can install the PostgreSQL packages using the following command:

```
$ sudo dnf install -y postgresql16-server postgresql16
```

Then you need to configure the system, specifying the PGDATA directory and enabling the option to start the service at boot time. In order to specify the PGDATA directory, you need to use `systemd(1)` to edit an overriding configuration file for the `postgresql-16` service:

```
$ sudo systemctl edit postgresql-16
```

The preceding command will open your default text editor with an empty file; you can, therefore, set the PGDATA variable as follows and then save and exit the editor to apply changes:

```
[Service]
Environment=PGDATA=/postgres/16/data
```

Lastly, it is time to initialize the database directory; this can be done with a specific Fedora installation command named `postgresql-16-setup`, as follows:

```
$ sudo /usr/pgsql-16/bin/postgresql-16-setup initdb
```

In order to enable PostgreSQL to start at boot time and launch the server immediately, you can execute the following commands:

```
$ sudo systemctl enable postgresql-16

$ sudo systemctl start postgresql-16.service
```

If your Fedora installation contains the `service(8)` command, you can also start the service with the following:

```
$ sudo service postgresql-16 start
```

Installing PostgreSQL on FreeBSD

PostgreSQL is available on FreeBSD by means of ports and packages. Thanks to the `pkg(1)` command, it is very easy to install PostgreSQL. First of all, update the package list, and search for the PostgreSQL packages that are named with the major version as the postfix:

```
$ pkg update
$ pkg search postgresql16
```

You can then install packages by executing `pkg(1)` and specify the set of packages you need. Of course, the installation must be executed as a user with administrative privileges, as follows:

```
$ sudo pkg install postgresql16-server-16.0 \
                  postgresql16-client-16.0 \
                  postgresql16-contrib-16.0 \
                  postgresql16-docs-16.0
```

In order to start the cluster, you need to initialize the directory to serve the database and enable the server startup at the machine boot. The minimal parameters to set are `postgresql_enable` and `postgresql_data`.

For example, to edit (as an administrative user) the `/etc/rc.conf` file, add the options as follows:

```
# to enable PostgreSQL at boot time
postgresql_enable="YES"

# PGDATA to use
postgresql_data="/postgres/16/data"
```

Now you can initialize the data directory with the following command:

```
$ sudo /usr/local/etc/rc.d/postgresql initdb
```

Now that everything is in place, you can start the PostgreSQL instance with the following command:

```
$ sudo service postgresql start
```

Installing PostgreSQL from sources

Installing PostgreSQL from sources requires downloading a tarball, which is a compressed package with all the source code files, and starting the compilation. Usually, this takes several minutes, depending on the power of the machine and the I/O bandwidth. In order to compile PostgreSQL from source, you will need different tools and libraries and mainly a C compiler compliant with the C99 standard (or higher). Usually, you already have these tools on a Linux or Unix system; otherwise, please refer to your operating system documentation on how to install these tools.

Once you have all the dependencies installed, follow the steps given here to compile and install PostgreSQL:

1. The very first step is to download the PostgreSQL tarball related to the version you want to install, verifying that it is correct. For instance, to download version 16.0, you can do the following:

```
$ wget https://ftp.postgresql.org/pub/source/v16.0/postgresql-16.0.tar.bz2
...
$ wget https://ftp.postgresql.org/pub/source/v16.0/postgresql-16.0.tar.bz2.md5
```

2. Before starting the compilation, check that the downloaded tarball is intact:

```
$ md5sum --check postgresql-16.0.tar.bz2.md5
postgresql-16.0.tar.bz2: OK
```